Kontrolltöö 2: kordamine

10IT · Sissejuhatus infotehnoloogiasse · kontrolltöö 17.11 · 45 minutit · 35 punkti · tunnid 5–9

Kuidas valmistuda

1. Tee proovitöö (lk 2–5) paberil läbi 45 minutiga, ilma konspektita. Prindi need lehed või kirjuta vastused vihikusse.
2. Kontrolli vastused lahendustest ja loe punktid kokku. Hinne 5 on alates 32 punktist, 4 alates 27, 3 alates 20.
3. Iga teema kohta, kus kaotasid punkte, loe meeldetuletus ja siis selle tunni konspekt. Lahendustes on kirjas, mis leheküljelt konspekt algab.
4. Tee kursuse lehel harjutus: samad ülesanded teiste arvudega, vihjete ja kohese kontrolliga.

Mis selles failis on

Osa	Mis seal on	Lk
Proovitöö	täpselt nagu kontrolltöö, ainult arvud ja sõnad on teised	2–5
Lahendused ja kordamine	iga teema lühike meeldetuletus ja proovitöö lahendused	6–8
5. tund: mis on arvuti sees?	konspekt: kogu teema algusest peale	9–13
6. tund: loogikaväravad	konspekt: kogu teema algusest peale	14–17
7. tund: protsessor	konspekt: kogu teema algusest peale	18–21
8. tund: pildid ja heli	konspekt: kogu teema algusest peale	22–28
9. tund: pakkimine	konspekt: kogu teema algusest peale	29–32

Lehekülgede numbrid on selle faili omad. Iga osa jalusel on osa enda numbrid.

Proovitöö: kontrolltöö 2

10IT · Sissejuhatus infotehnoloogiasse · proovitöö · 45 minutit · kokku 35 punkti

Nimi		Kuupäev		**Proovitöö**

See on proovitöö. Päris kontrolltöö 17.11 on täpselt samasugune: samad ülesanded, samad punktid ja sama ülesehitus, ainult arvud ja sõnad on teised. Tee see läbi 45 minutiga ilma konspektita, siis kontrolli vastuseid lahenduste failist ja korda need teemad, kus said vähem punkte.

Kirjuta vastused otse töölehele. Kalkulaatorit ei kasuta, mänguprotsessori käsutabel on 5. ülesandes. Näita arvutustes ka vahesamme: kui meetod on õige ja viga on ainult arvutuses, saad enamiku punktidest. Kui mõni ülesanne ei tule välja, mine edasi ja tule lõpus tagasi.

1. Mõisted (5 p)

Iga mõiste kohta on all üks selgitus. Kirjuta lahtrisse õige täht.

Mõiste	Täht
RAM	
tuum	
register	
piksel	
kadudega pakkimine	

A. osa infot visatakse jäädavalt ära, näiteks JPEG

B. töömälu: kiire, aga kaotab sisu, kui vool kaob

C. üks protsessor protsessori sees, mitu tuuma saavad töötada korraga

D. pildi väikseim ruut, mis on ühte värvi

E. protsessori enda pisike ja väga kiire mälu, mängus A

2. Arvuti sees (4 p)

a) Järjesta kiiremast aeglasemani: SSD, register, RAM, vahemälu. (1 p)

b) Kuulutus: `Ryzen 5 7600 · 16 GB DDR5 · 1 TB NVMe SSD · RTX 4060 8 GB`. Mitu GB on selles arvutis RAM-i? (1 p)

c) AM5 pesaga protsessor ja LGA1700 pesaga emaplaat. Kas need töötavad koos? Põhjenda. (1 p)

d) Kirjutad referaati ja arvuti jookseb enne salvestamist kokku. Miks on töö kadunud? (1 p)

3. Loogikaväravad (7 p)

a) Täida tabelis väravate väljundid. Iga õigesti täidetud veerg annab 1 punkti. (4 p)

A	B	A AND B	A OR B	A XOR B	A NAND B
0	0				
0	1				
1	0				
1	1				

b) Arvuta väljund. Tee seda samm-sammult: kõigepealt sulgudes olev osa. (2 p)

Skeem	Sisendid	Väljund
`NOT (A AND B)`	A = 1, B = 1	
`(A OR B) AND NOT C`	A = 1, B = 0, C = 1	

c) Mitu rida on tõeväärtustabelis, kui sisendeid on 4? (1 p)

4. Liitmine (3 p)

a) Liida kahendsüsteemis tulbas: `0101` + `0110`. Kirjuta vastus kahend- ja kümnendsüsteemis. (2 p)

kahendsüsteemis

kümnendsüsteemis

Tulbas liitmine:

b) Milline värav annab pooliitjas summa S? (1 p)

5. Protsessor (6 p)

Mänguprotsessoril on 16 mälupesa (0–15) ja register A. Iga käsk on üks bait: esimene 16-süsteemi number ütleb, mida teha, teine, millise pesaga.

Kood	Käsk	Mida teeb	Kood	Käsk	Mida teeb
`0_`	`HALT`	peatab programmi	`4x`	`STORE x`	kirjutab A väärtuse pessa x
`1x`	`LOAD x`	A = pesa x sisu	`5_`	`OUT`	paneb A väärtuse väljundisse
`2x`	`ADD x`	A = A + pesa x sisu	`6x`	`JUMP x`	järgmine käsk tuleb pesast x
`3x`	`SUB x`	A = A − pesa x sisu	`7x`	`JZ x`	kui A on 0, järgmine käsk tuleb pesast x

a) Mis käsk on bait `2E`? Kirjuta assembleris. (1 p)

b) Kirjuta käsk `STORE 12` 16-süsteemi baidina. (1 p)

c) Mälus on see programm. Alguses on A = 0. Täida tabel: mis on A pärast igat takti ja mis arvu see takt väljundisse pani (kui ei pannud, kirjuta kriips). Esimene rida on tehtud. (3 p)

```
0  LOAD 13
1  ADD 14
2  SUB 15
3  OUT
4  HALT
   ...
13 DAT 9
14 DAT 4
15 DAT 2
```

Takt	Käsk	A pärast	Väljund
1	`LOAD 13`	9	–
2	`ADD 14`		
3	`SUB 15`		
4	`OUT`		

d) Protsessori taktsagedus on 4 GHz. Mida see tähendab? (1 p)

6. Pildid ja heli (5 p)

RGB-pildis võtab iga piksel 3 baiti. CD-heli: 44 100 mõõtmist sekundis, 2 baiti mõõtmise kohta. 1 MB = 1 000 000 baiti.

a) Mustvalge pilt on 16 × 16 pikslit, igal pikslil 1 bitt. Mitu baiti võtavad pikslid? (1 p)

b) Kirjuta pildi rida `01111110` 16-süsteemis. (1 p)

c) Kirjuta värvikoodina RGB (255, 0, 128). (1 p)

d) RGB-pilt on 2 000 pikslit lai ja 1 000 kõrge. Kui suur on selle pakkimata andmemaht megabaitides? Näita arvutuskäiku. (1 p)

e) Mitu baiti võtab 5 sekundit pakkimata CD-heli stereos? Näita arvutuskäiku. (1 p)

7. Pakkimine (5 p)

RLE: V on valge, M on must. Pakkimata võtab iga piksel 1 baidi, pakituna võtab iga paar 2 baiti (näiteks `5V` või `10M`).

a) Paki RLE-ga rida `VVVMMMMMMV`. Mitu baiti see pakituna võtab? (1 p)

b) Paki lahti `2M4V2M`. (1 p)

c) Fail oli 10 MB, pärast pakkimist 2 MB. Mitu korda väiksemaks see jäi ja mitu protsenti säästeti? Näita arvutuskäiku. (2 p)

d) Ekraanipildil on palju teksti. Kas salvestad selle PNG-na või JPEG-ina? Miks? (1 p)

Kontrolli enne ära andmist, et nimi on lehel ja ükski ülesanne ei jäänud kogemata vahele. Edu!

Proovitöö lahendused ja kordamine

10IT · Sissejuhatus infotehnoloogiasse · kontrolltöö 2 · 17.11

Kontrolltöö 17.11 on täpselt sama ülesehitusega nagu proovitöö: 7 ülesannet, samad punktid, kokku 35 punkti, 45 minutit. Muutuvad ainult arvud ja sõnad. Iga ülesande juures on lühike meeldetuletus, mida pead oskama, ja lahendus koos käiguga. Hinne 5 on alates 32 punktist, 4 alates 27, 3 alates 20.

1. Mõisted (5 p)

> Mõisted tulevad 5.–9. tunni sõnastikest. Korda konspektide viimaseid lehekülgi: seal on iga tunni mõisted koos lühikese selgitusega.

Ülesanne	Lahendus	P
1. Mõisted	RAM B, tuum C, register E, piksel D, kadudega pakkimine A	5

2. Arvuti sees (4 p)

> **Meeldetuletus (5. tund).** Arvuti on köök: protsessor on kokk, RAM on tööpind, SSD on kelder ja emaplaat on köök ise. Mäluredel kiiremast aeglasemani: register, vahemälu, RAM, SSD, kõvaketas, võrk. Mida kiirem mälu, seda kallim ja väiksem. RAM kaotab sisu, kui vool kaob, ketas mitte. Osad peavad sobima: protsessori pesa (AM4, AM5, LGA1700) ja mälutüüp (DDR4 või DDR5) peavad emaplaadiga klappima. F-tähega Inteli protsessoril pole graafikat. Kuulutuses on videokaardi „8 GB“ tema enda mälu, mitte RAM. Loe üle: 5. tunni konspekt „Mis on arvuti sees?“, selle faili lk 9.

Ülesanne	Lahendus	P
2a) järjestus	register, vahemälu, RAM, SSD	1
2b) RAM	16 GB (videokaardi mälu ja SSD ei ole RAM)	1
2c) kas töötavad koos	Ei sobi: protsessori pesa (socket) ja emaplaadi pesa peavad olema samad.	1
2d) Kirjutad referaati ja arvuti jookseb enne salvestamist kokku. Miks on töö kadunud?	Salvestamata töö oli ainult RAM-is. RAM kaotab sisu, kui vool kaob. Alles jääb ainult see, mis on salvestatud kettale (SSD).	1

3. Loogikaväravad (7 p)

> **Meeldetuletus (6. tund).** AND: 1 ainult siis, kui mõlemad on 1. OR: 1, kui vähemalt üks on 1. NOT pöörab ümber. XOR: 1, kui sisendid on erinevad. NAND ja NOR on AND ja OR ümberpööratult. Tabeli read käivad alati järjekorras 00, 01, 10, 11. Iga lisasisend kahekordistab ridade arvu: n sisendiga on ridu 2 astmel n. Loe üle: 6. tunni konspekt „Loogikaväravad“, selle faili lk 14.

Ülesanne	Lahendus	P
3a) A AND B	0 0 0 1 (ridades 00, 01, 10, 11)	1
3a) A OR B	0 1 1 1 (ridades 00, 01, 10, 11)	1
3a) A XOR B	0 1 1 0 (ridades 00, 01, 10, 11)	1
3a) A NAND B	1 1 1 0 (ridades 00, 01, 10, 11)	1
3b) NOT (A AND B), A = 1, B = 1	0	1
3b) (A OR B) AND NOT C, A = 1, B = 0, C = 1	0	1
3c) ridu 4 sisendiga	16 (2 astmel 4)	1

4. Liitmine (3 p)

> **Meeldetuletus (6. tund).** Kahendsüsteemis tulbas liites: 0 + 0 = 0, 0 + 1 = 1, 1 + 1 = 10 (kirjuta 0, 1 meelde), 1 + 1 + 1 = 11. Pooliitjas annab summa S XOR ja ülekande C AND. Loe üle: 6. tunni konspekt „Loogikaväravad“, selle faili lk 14.

Ülesanne	Lahendus	P
4a) 0101 + 0110	1011 ehk 11 (5 + 6)	2
4b) Milline värav annab pooliitjas summa S?	XOR	1

5. Protsessor (6 p)

> **Meeldetuletus (7. tund).** Bait on kaks 16-süsteemi numbrit: esimene ütleb käsu, teine pesa. LOAD 14 = 1 ja E = 1E. Protsessor teeb igal ringil sama: too, dekodeeri, täida. PC näitab, kust tuleb järgmine käsk. 4 GHz on 4 miljardit takti sekundis. Loe üle: 7. tunni konspekt „Kuidas protsessor programmi täidab“, selle faili lk 18.

Ülesanne	Lahendus	P
5a) 2E	ADD 14	1
5b) STORE 12	4C	1
5c) takt 2: ADD 14	A = 13, väljund –	1
5c) takt 3: SUB 15	A = 11, väljund –	1
5c) takt 4: OUT	A = 11, väljund 11	1
5d) Protsessori taktsagedus on 4 GHz. Mida see tähendab?	Protsessori kell lööb 4 miljardit korda sekundis: 4 miljardit takti sekundis. Giga on miljard, herts on kord sekundis.	1

6. Pildid ja heli (5 p)

> **Meeldetuletus (8. tund).** Mustvalges pildis on 1 bitt pikslis: baite = laius × kõrgus : 8. 16-süsteemi jaoks jaga 8 bitti pooleks ja kirjuta kumbki pool üheks märgiks. Värvikood on R, G ja B 16-süsteemis, kumbki kaks märki. Pakkimata RGB-pilt = laius × kõrgus × 3 baiti. Pakkimata heli = mõõtmisi sekundis × baite mõõtmises × kanaleid × sekundeid. Loe üle: 8. tunni konspekt „Pildid ja heli arvudena“, selle faili lk 22.

Ülesanne	Lahendus	P
6a) 16 × 16, 1 bitt	16 × 16 = 256 bitti, : 8 = 32 baiti	1
6b) 01111110	0111 1110 → 7E	1
6c) RGB (255, 0, 128)	#FF0080	1
6d) 2 000 × 1 000 RGB	2 000 × 1 000 × 3 = 6 000 000 baiti = 6 MB	1
6e) 5 s, stereo	44 100 × 2 × 2 × 5 = 882 000 baiti	1

7. Pakkimine (5 p)

> **Meeldetuletus (9. tund).** RLE loeb kokku, mitu ühesugust on järjest, iga paar on 2 baiti. Mitu korda väiksem = enne : pärast. Säästetud % = (enne − pärast) : enne × 100. Kadudeta (PNG, ZIP) annab täpselt sama faili tagasi, kadudega (JPEG, MP3) viskab infot ära. Juba pakitud faili uuesti pakkida ei tasu. Loe üle: 9. tunni konspekt „Pakkimine“, selle faili lk 29.

Ülesanne	Lahendus	P
7a) VVVMMMMMMV	3V 6M 1V, 6 baiti	1
7b) 2M4V2M	MMVVVVMM	1
7c) 10 MB → 2 MB	10 : 2 = 5 korda; (10 − 2) : 10 × 100 = 80 %	2
7d) Ekraanipildil on palju teksti. Kas salvestad selle PNG-na või JPEG-ina? Miks?	PNG. See pakib kadudeta, nii et tähtede teravad servad jäävad teravaks. JPEG teeks tähtede ümber plekilise aura.	1

Lahenduskäigud lähemalt

3b. NOT (A AND B), kui A = 1 ja B = 1: kõigepealt sulud, 1 AND 1 = 1. Siis NOT 1 = 0. (A OR B) AND NOT C, kui A = 1, B = 0, C = 1: A OR B = 1, NOT C = 0, 1 AND 0 = 0.

4a. `0101` + `0110`: paremalt 1 + 0 = 1, 0 + 1 = 1, 1 + 1 = 10 (kirjuta 0, 1 meelde), 0 + 0 + 1 = 1. Vastus `1011`. Kontroll: 5 + 6 = 11 ja 8 + 2 + 1 = 11.

5a ja 5b. `2E`: 2 on ADD, E on 14, seega ADD 14. STORE on 4 ja 12 on C, seega `4C`.

5c. Takt 1: LOAD 13 paneb A-sse pesa 13 sisu, 9. Takt 2: ADD 14, 9 + 4 = 13. Takt 3: SUB 15, 13 − 2 = 11. Takt 4: OUT näitab A-d, väljundisse tuleb 11, A jääb 11.

6b. 0111 1110: 0111 on 7 ja 1110 on 14 ehk E, seega 7E. **6c.** 255 = FF, 0 = 00, 128 = 8 · 16 + 0 = 80, seega #FF0080.

6e. Ühe sekundi üks kanal on 44 100 × 2 = 88 200 baiti. Stereo on kaks kanalit: 176 400. Viis sekundit: 176 400 × 5 = 882 000 baiti ehk umbes 0,9 MB.

7a. `VVVMMMMMMV`: kolm valget, kuus musta, üks valge: `3V 6M 1V`. Kolm paari, igaüks 2 baiti, kokku 6 baiti 10 asemel.

Mis on arvuti sees?

10IT · Sissejuhatus infotehnoloogiasse · 5. tund · konspekt

> **Selle tunni suur mõte:** arvuti teeb kogu aeg üht ja sama: võtab andmed, teeb nendega midagi ja paneb tulemuse tagasi. Peaaegu iga osa korpuse sees on olemas selleks, et andmed jõuaksid protsessorini kiiremini.

Arvuti on köök

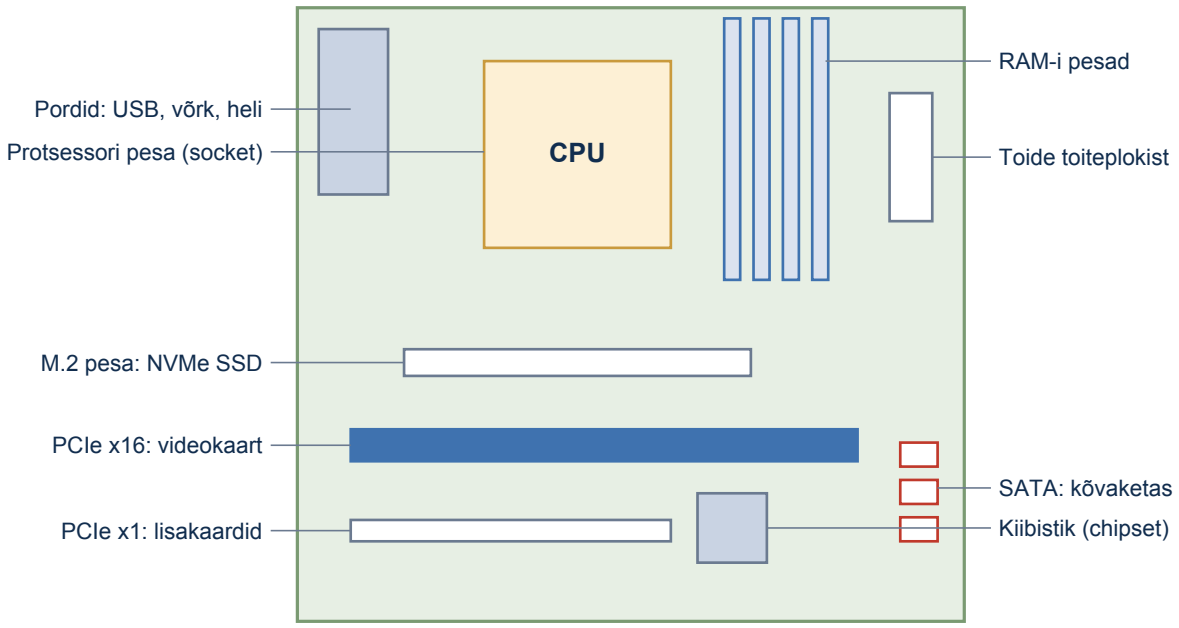
Kujuta ette restorani kööki õhtusel tipptunnil. Kokk teeb toitu, aga ta ei jookse iga sibula järele keldrisse. Tal on tööpind, kuhu ta tõstab kõik, mida järgmise poole tunni jooksul vaja läheb. Keldris on varud, mida ei ole veel vaja. Kõike seda ühendab köök ise: põrand, torud, juhtmed ja elekter.

Arvuti on ehitatud täpselt samamoodi. Seda võrdlust kasutame kogu tunni jooksul, nii et jäta see meelde:

Osa	Mida teeb	Köögis
Protsessor (CPU)	Teeb kõik arvutused ja käsud, ühe teise järel, väga kiiresti.	Kokk
Mälu (RAM)	Hoiab seda, millega praegu töötatakse: lahtised programmid, failid, brauseri vahelehed.	Tööpind
Ketas (SSD või HDD)	Hoiab kõike pikaajaliselt: programme, pilte, Windowsit. Ka siis, kui arvuti on välja lülitatud.	Kelder
Emaplaat	Suur trükkplaat, mille külge kõik teised osad ühenduvad.	Köök ise: põrand, torud, juhtmed
Videokaart (GPU)	Joonistab pilti ekraanile. Tal on tuhandeid väikseid tuumasid, mis teevad lihtsaid arvutusi korraga.	Tuhat abikokka, kes oskavad ainult porgandit hakkida
Toiteplokk (PSU)	Teeb pistikupesa 230 voldist väiksemad pinged, mida arvuti osad vajavad.	Elekter ja gaas
Jahutus ja korpus	Viib soojuse minema ja hoiab kõike koos.	Õhupuhasti ja seinad

Vaatame emaplaati

Kui võtad lauaarvuti küljekaane ära, näed kõigepealt emaplaati. Kõik teised osad on selle külge kinnitatud või kaabliga ühendatud. Pildil on lihtsustatud emaplaat ülevalt vaadates.



Kõige tähtsam asi, mida emaplaadi kohta teada: **osad peavad sobima**. Protsessoril on kindel pesa (socket), näiteks AMD-l **AM4** või **AM5** ja Intelil **LGA1700**. AM5 protsessor ei lähe LGA1700 emaplaadile, täpselt nagu Euroopa pistik ei lähe Briti pistikupessa. Sama lugu on mäluga: emaplaat toetab kas **DDR4** või **DDR5** mälu, mitte mõlemat. Pulgad on isegi eri kohast sälguga, et neid valesse pessa ei saaks suruda.

> **Tähelepanu:** „sobib“ on arvuti kokkupanemisel kõige tähtsam sõna. Kalleim protsessor on väärtusetu, kui see emaplaadile ei lähe.

Protsessor lähemalt

Protsessor on kokk. Ta loeb retsepti (programmi) rida rea haaval ja teeb täpselt seda, mis kirjas. Iga rida on väga lihtne: liida kaks arvu, võrdle kaht arvu, too mälust üks arv. Protsessor ei ole tark. Ta on lihtsalt meeletult kiire.

Taktsagedus: mitu sammu sekundis

Protsessori sees tiksub kell. Iga tiksuga saab ta teha ühe sammu. Taktsagedust mõõdetakse hertsides: 1 Hz on üks tiks sekundis. Tänapäeva protsessoril on tavaliselt 3 kuni 5 GHz ehk 3 kuni 5 **miljardit** tiksu sekundis.

Kui palju see on? Valgus jõuab ühe sekundiga seitse ja pool korda ümber maakera. 4 GHz protsessori ühe tiksu ajal jõuab valgus liikuda umbes 7,5 cm, ehk vähem kui sinu peopesa laius.

Tuumad: mitu kokka köögis

Umbes 2005. aastal jäi taktsagedus 3–4 GHz juurde kinni. Kiirem kell tähendas rohkem soojust ja protsessorid oleksid lihtsalt üle kuumenenud. Insenerid tegid teise asja: panid ühte kiipi mitu protsessorit. Igaüht neist nimetatakse **tuumaks** (core). Tänapäeva protsessoris on tavaliselt 4 kuni 16 tuuma.

Mitu kokka saavad korraga valmistada mitu rooga. Aga ühe supi keetmine ei lähe kaheksa kokaga kaheksa korda kiiremini: kõigepealt peab vesi keema minema ja seda ei saa jagada. Arvutiga on sama lugu. Video eksportimine jaguneb kenasti kõigi tuumade vahel (iga tuum teeb oma tüki videost), aga paljud mängud ja tavaprogrammid kasutavad korraga ainult paari tuuma.

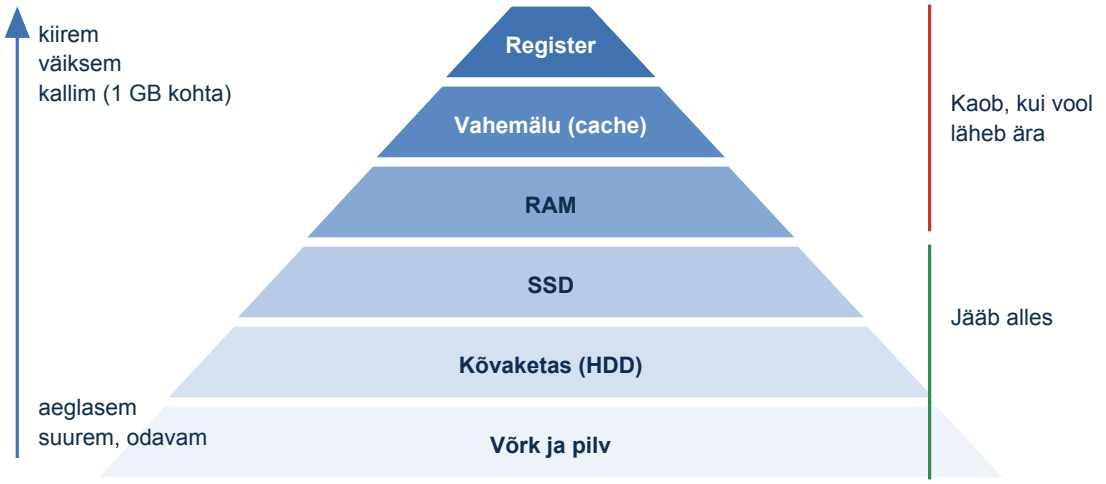
Rusikareegel: mängude jaoks loeb rohkem kiire tuum (kõrge GHz) ja hea videokaart. Videomontaaži ja 3D jaoks loeb rohkem tuumade arv.

Vahemälu ehk cache

Protsessori kõrval on ka väike ja väga kiire mälu, mida kutsutakse vahemäluks (cache). Seal on mõned megabaidid andmeid, mida protsessor kasutas just praegu ja vajab tõenäoliselt kohe uuesti. Köögis oleks see lõikelaud otse koka ees. Tööpinnale (RAM) on samm, lõikelauale ei ole sedagi.

Mäluhierarhia: mis on lähedal, mis kaugel

Miks on arvutis nii palju eri mälusid? Miks mitte teha kogu mälu sama kiireks kui vahemälu? Vastus on raha ja füüsika: mida kiirem mälu, seda kallim ja seda vähem seda protsessori kõrvale mahub. Nii tekib redel, mille ülemisel pulgal on vähe ja kiiret mälu ning alumisel palju ja aeglast.



Päris ajad on nii lühikesed, et neid on raske ette kujutada. Seepärast on tabelis ka teine tulp: kujutame ette, et protsessori registrist arvu lugemine võtaks **ühe sekundi**, ja venitame kõik teised ajad samas suhtes pikemaks.

Kus andmed on	Kui palju mahub	Päriselt	Kui register oleks 1 s	Köögis
Register	mõnisada baiti	alla 1 ns	1 sekund	Käes
Vahemälu (cache)	mõni MB	1–10 ns	1–10 sekundit	Lõikelaud
RAM	8–64 GB	umbes 100 ns	poolteist minutit	Tööpind
SSD	0,5–4 TB	umbes 0,1 ms	üle ühe päeva	Kelder
Kõvaketas (HDD)	1–20 TB	umbes 10 ms	umbes 4 kuud	Naabri kelder
Server teisel mandril	lõputult	umbes 100 ms	umbes 3 aastat	Pood teisel mandril

ns on nanosekund, sekundi miljardik osa. ms on millisekund, sekundi tuhandik osa.

Nüüd on näha, miks arvuti hakkab tohutult aeglaseks minema, kui protsessor peab andmeid kettalt ootama. Minuti asemel kulub päev. Ja miks veebileht laeb silmnähtavalt kauem kui fail sinu arvutis: seal tuleb oodata „aastaid“.

Miks RAM ja SSD ei ole sama asi

Telefoni kuulutuses on kirjas „8 GB / 256 GB“. Paljud ütlevad mõlema kohta „mälu“, aga need on täiesti erinevad asjad. 8 GB on RAM ehk tööpind. 256 GB on salvestusruum ehk kelder.

	RAM	SSD
Mis seal on	Praegu lahti olevad programmid ja failid	Kõik failid ja programmid, ka need, mida praegu ei kasutata
Kiirus	Umbes tuhat korda kiirem kui SSD	Aeglasem, aga palju kiirem kui vana kõvaketas
Kui vool kaob	Kõik kaob	Kõik jääb alles
Tavaline suurus	8–32 GB	256 GB – 2 TB
Hind 1 GB kohta (umbes)	3 €	0,06 €

Mäletad, kui kirjutasid referaati ja arvuti jooksis kokku enne salvestamist? Kõik, mida kirjutasid, oli ainult RAM-is. **Salvestamine** tähendabki seda, et fail kopeeritakse RAM-ist kettale, kus see jääb alles ka pärast voolu kadumist.

Mis juhtub, kui RAM saab täis? Oled ehk näinud, kuidas sada lahtist Chrome'i vahelehte teeb arvuti nätskeks. Kui tööpinnal ruumi pole, tõstab operatsioonisüsteem osa asju kettale ja toob need vajadusel tagasi. Arvuti töötab edasi, aga iga kord, kui tuleb midagi kettalt tagasi tuua, kulub minuti asemel päev. Seepärast tundub RAM-i juurde panemine vahel kiirendavat rohkem kui uus protsessor.

Kuidas lugeda arvuti kuulutust

Nüüd peaksid oskama lugeda sellist kirjeldust, mida näed igas arvutipoes:

```
Ryzen 5 7600 · 32 GB DDR5 · 1 TB NVMe SSD · RTX 4060 8 GB · 650 W
```

Kirjas	Mida see tähendab
Ryzen 5 7600	AMD protsessor, 6 tuuma, kuni 5,1 GHz. Pesa AM5, seega emaplaat peab olema AM5.
32 GB DDR5	RAM: 32 GB, tüüp DDR5. Emaplaat peab toetama DDR5 mälu.
1 TB NVMe SSD	Salvestusruum 1 TB. NVMe tähendab kiiret SSD-d, mis läheb emaplaadi M.2 pessa.
RTX 4060 8 GB	NVIDIA videokaart. 8 GB on videokaardi enda mälu, mitte arvuti RAM.
650 W	Toiteplokk, mis suudab anda kuni 650 vatti. Peab olema suurem, kui osad kokku tarbivad.

Üks lõks veel: kõigil protsessoritel ei ole sisseehitatud graafikat. Näiteks Inteli protsessorid, mille nime lõpus on F-täht (i5-12400**F**), ja vanemad AMD Ryzenid ilma G-täheta vajavad eraldi videokaarti. Ilma selleta jääb ekraan lihtsalt mustaks.

Tunni teises pooles: pane ise arvuti kokku

Teises pooles paned kokku kolm arvutit kolmele väga erinevale inimesele, igaühe eelarve on 800 €. Selleks kasutad lehte **PCPartPicker**, kus on päris osad päris hindadega. Leht kontrollib ise, kas osad sobivad kokku, ja arvutab, mitu vatti arvuti vajab. Iga valmis arvuti lingi ja põhjenduse kirjutad tunnitöö lehele.

Pea meeles, et 800 € ei pea ära kulutama. Kui keegi vajab ainult e-kirju ja videokõnesid, on kallis videokaart raha raiskamine.

Mõisted

Mõiste	Lühidalt
protsessor (CPU)	teeb arvutused ja täidab programmi käske
tuum (core)	üks protsessor protsessori sees; mitu tuuma saavad töötada korraga
taktsagedus (GHz)	mitu miljardit sammu teeb protsessor sekundis
vahemälu (cache)	väike ja väga kiire mälu protsessori sees
RAM	töömälu; kiire, aga kaotab sisu, kui vool kaob
SSD / HDD	püsimälu ehk ketas; SSD on kiibid, HDD on pöörlevad plaadid
emaplaat	plaat, mis ühendab kõik osad; määrab, milline protsessor ja mälu sobivad
pesa (socket)	protsessori pesa emaplaadil, nt AM4, AM5, LGA1700
videokaart (GPU)	joonistab pilti; palju lihtsaid tuumasid
toiteplokk (PSU)	annab arvuti osadele voolu; mõõdetakse vattides (W)

Loogikaväravad

10IT · Sissejuhatus infotehnoloogiasse · 6. tund · konspekt

Kuidas protsessor liidab?

Eelmises tunnis panid kokku arvuteid ja valisid neile protsessori. Selle sees on miljardeid transistore. Aga mida nad seal õigupoolest teevad?

Võtame lihtsa ülesande: 5 + 3. Kahendsüsteemis on see `0101 + 0011`. Protsessori sees ei ole kedagi, kes teaks, mis on „viis“. Seal on ainult juhtmed ja igas juhtmes kas on vool või ei ole. Üks või null. Kuidas sellisest asjast saab masin, mis oskab arvutada? Täna paneme selle kokku, alates kõige väiksemast tükist.

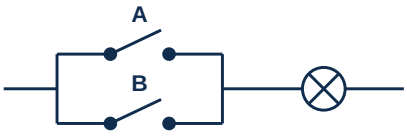
Transistor on lüliti

Transistor on pisike lüliti, mida ei keera näpp, vaid elekter. Kui juhtsisendis on vool, laseb transistor voolu läbi, ja kui ei ole, siis ei lase. Vool on 1, voolu puudumine on 0. Rohkem ta ei oska.

Huvitavaks läheb siis, kui panna mitu lülitit koos tööle. Vaata kahte lambiga vooluringi:



järjest: lamp põleb, kui A JA B on sees

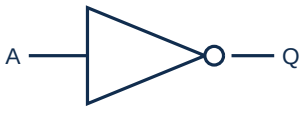


kõrvuti: lamp põleb, kui A VÕI B on sees

Vasakul on lülitid järjest ja lamp põleb ainult siis, kui mõlemad on sees. Paremal on nad kõrvuti ja lambile piisab ühest. Need kaks vooluringi on juba **loogikaväravad**. Värav on väike skeem, mis võtab sisse bitte ja annab välja ühe biti kindla reegli järgi.

Reegli saab kirja panna tabelina, kus on kõik võimalikud sisendid ja iga sisendi juures vastus. Sellist tabelit nimetatakse **tõeväärtustabeliks**. Kahe sisendi puhul on võimalusi neli: 00, 01, 10 ja 11.

Kolm põhiväravat



EI (NOT)



JA (AND)



VÕI (OR)

EI (NOT): pöörab ümber

Kõige lihtsam värav, ainult ühe sisendiga. Mis sisse läheb, see tuleb vastupidi välja. Väike ring sümboli otsas tähendab alati „ümber pööratud“.

JA (AND): mõlemad peavad olema

Väljund on 1 ainult siis, kui **kõik** sisendid on 1. Kooli välisuks avaneb, kui näitad kaarti *ja* sisestad PIN-koodi. Ühest üksi ei piisa.

VÕI (OR): piisab ühest

Väljund on 1, kui **vähemalt üks** sisend on 1. Häire läheb tööle, kui avatakse uks *või* aken, ja muidugi ka siis, kui mõlemad.

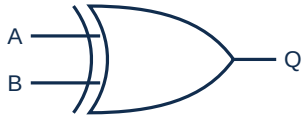
A	NOT A
0	1
1	0

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

> **Lõks:** igapäevakeeles tähendab „või“ tihti „üks või teine, aga mitte mõlemad“. „Kas teed või kohvi?“ ei tähenda, et võid võtta mõlemat. Loogikas on VÕI helde: 1 VÕI 1 on 1. Range „kas üks või teine“ on hoopis eraldi värav, XOR.

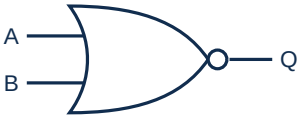
XOR, NAND ja NOR



XOR (välistav VÕI)



NAND (JA-EI)



NOR (VÕI-EI)

XOR annab 1, kui sisendid on **erinevad**. Täpselt üks peab olema sees. Mõtle trepivalgustusele, kus lülitid on all ja üleval: kumba lülitit ükskõik millal vajutad, lambi olek muutub. See on XOR.

NAND on JA, mille väljund on ümber pööratud, ja **NOR** on ümber pööratud VÕI. Neid on lihtne unustada, sest nad ei tundu esimese hooga kasulikud. Tegelikult on NAND üks olulisemaid väravaid üldse: ainult NAND-väravatest saab kokku panna **kõik** teised väravad ja seega terve protsessori. Sinu mälupulga mälu nimetatakse muide *NAND flash*.

A	B	XOR	NAND	NOR
0	0	0	1	1
0	1	1	1	0
1	0	1	1	0
1	1	0	0	0

Väravad koos

Ühe värava väljundi saab ühendada teise sisendiga. Nii tekivad skeemid, mis oskavad juba päris asju otsustada. Proovime: `(A AND B) OR NOT C`. Kolme sisendiga on võimalusi kaheksa, ja kõige kindlam on need läbi käia nii, nagu loendaksid kahendsüsteemis: 000, 001, 010 ja nii edasi kuni 111. Siis ei jää ükski rida vahele.

Kõigepealt arvuta vahetulemused eraldi veergu ja alles siis lõppvastus. Peast korraga tehes läheb peaaegu kindlasti mõni rida sassi.

A	B	C	A AND B	NOT C	tulemus
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

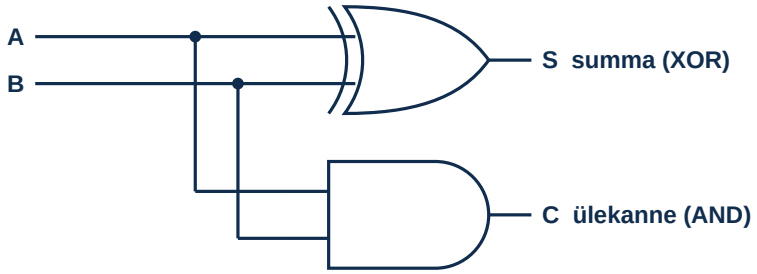
Pane tähele: sisendeid on 3 ja ridu $2^3 = 8$. Nelja sisendiga oleks ridu 16. See on täpselt sama reegel nagu esimeses tunnis: iga lisabitt kahekordistab võimaluste arvu.

Pooliitja: masin, mis liidab

Nüüd tagasi küsimuse juurde, millega alustasime. Kuidas liita kaks bitti? Võimalusi on neli:

A	B	A + B	ülekanne C	summa S
0	0	00	0	0
0	1	01	0	1
1	0	01	0	1
1	1	10	1	0

Vaata kahte viimast veergu eraldi. Summa S on 1 siis, kui bitid on erinevad, nii et see on XOR. Ülekanne C on 1 ainult siis, kui mõlemad on 1, ehk AND. Kogu liitmine on kaks väravat:



Seda skeemi nimetatakse **pooliitjaks**. „Pool“ sellepärast, et ta ei oska arvestada eelmisest veerust tulnud ülekannet. Kui lisada veel paar väravat, saab **täissummaatori**, millel on kolm sisendit: A, B ja eelmise veeru ülekanne.

Pane neli täissummaatorit ritta ja iga veeru ülekanne läheb järgmisele edasi, täpselt nagu koolis tulbas liites „üks meelde“. Ongi neljabitine liitja:

```
ülekanne    1 1 1
              0 1 0 1   (5)
          + 0 0 1 1   (3)
            -------
            1 0 0 0   (8)
```

Sinu arvuti 64-bitine protsessor teeb sedasama, ainult 64 veergu korraga ja mitu miljardit korda sekundis.

Kui palju neid on?

Üks täissummaator on umbes 28 transistorit. Uues telefoni- või arvutiprotsessoris on transistore **kümneid miljardeid**. Kõik, mida arvuti teeb, alates mängudest ja lõpetades videotega, on lõpuks nendest pisikestest JA-, VÕI- ja EI-väravatest kokku pandud. Ükski neist ei ole tark. Nutikus tekib sellest, kuidas nad on kokku ühendatud.

Mõiste	Tähendus
transistor	elektriga juhitav lüliti, protsessori kõige väiksem ehitusplokk
loogikavärav	skeem, mis teeb sisendbittidest reegli järgi ühe väljundbiti
tõeväärtustabel	tabel, kus on kõik sisendite kombinatsioonid ja igaühe väljund
AND, OR, NOT	JA, VÕI, EI: kolm põhiväravat
XOR	1, kui sisendid on erinevad
NAND, NOR	ümberpööratud AND ja OR
pooliitja	XOR + AND: liidab kaks bitti, annab summa ja ülekande

Kuidas protsessor programmi täidab

10IT · Sissejuhatus infotehnoloogiasse · 7. tund · konspekt

Liitja oskab liita. Aga kes vajutab nuppe?

Eelmises tunnis ehitasime väravatest liitja. Anna talle kaks arvu ja ta liidab need. Kalkulaator töötab umbes nii: sina vajutad 5, pluss, 3, võrdub ja tema arvutab.

Arvuti teeb aga palju rohkem. Ta mängib videot, kontrollib õigekirja, liigutab mängus tegelast. Ja keegi ei istu tema sees nuppe vajutamas. Kust ta siis teab, mida järgmiseks teha?

Vastus on **programm**. Programm on nimekiri käskudest: „võta see arv“, „liida juurde see“, „näita tulemust“. Protsessor loeb käske ükshaaval ja teeb täpselt seda, mis kirjas. Huvitav on see, kus see nimekiri asub ja mis kujul. Tuleb välja, et ka käsud on lihtsalt arvud.

Mälu on rida nummerdatud kappe

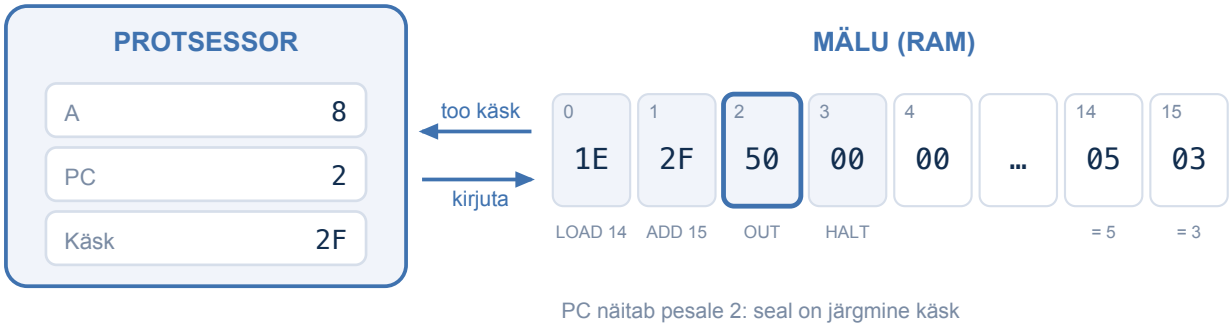
Mõtle koolikoridori riidekappidele. Igal kapil on number ja igas kapis on üks asi. Arvuti mälu (RAM, mida valisid 5. tunnis) on täpselt selline, ainult kappe on miljardeid. Igas kapis on üks bait ehk arv 0 kuni 255, 16-süsteemis 00 kuni FF.

Kapi numbrit nimetatakse **aadressiks**. Kui protsessor tahab mingit arvu, ütleb ta mälule aadressi: „anna mulle pesa 14 sisu“. Mälu ei tea, mis seal on: arv, täht, pildi täpp või käsk. Ta lihtsalt annab baidi.

Meie mänguprotsessor

Päris protsessor on liiga keeruline, et seda tunniga läbi vaadata. Seepärast on kursuse lehel pisike mänguprotsessor. See on ehitatud samade põhimõtete järgi, ainult palju väiksem. Tal on:

- **16 mälupesa**, aadressid 0 kuni 15;
- **register A**. See on protsessori enda pisike mälu, üks arv, millega ta parasjagu töötab. Nagu arv, mida hoiad peast arvutades meeles;
- **PC** (ingl *program counter*) ehk järjehoidja. See on aadress, kust tuleb järgmine käsk.



Protsessor ja mälu. Protsessor võtab mälust käske ja andmeid ning kirjutab tulemusi tagasi.

Käsud on arvud

Mänguprotsessor oskab ainult kaheksat käsku. See on vähe, aga nagu näed, piisab neist isegi korrutamiseks.

Kood	Käsk	Mida teeb
`0_`	`HALT`	peatab programmi
`1x`	`LOAD x`	A = pesa x sisu
`2x`	`ADD x`	A = A + pesa x sisu
`3x`	`SUB x`	A = A − pesa x sisu
`4x`	`STORE x`	kirjutab A väärtuse pessa x
`5_`	`OUT`	paneb A väärtuse väljundisse (ekraanile)
`6x`	`JUMP x`	järgmine käsk tuleb pesast x
`7x`	`JZ x`	kui A on 0, järgmine käsk tuleb pesast x. Muidu lähe edasi

Vaata koodi veergu. Iga käsk on üks bait, kaks 16-süsteemi numbrit. **Esimene number ütleb, mida teha, teine ütleb, millise pesaga.** `LOAD 14` on `1` (LOAD) ja `E` (14), kokku `1E`. Siin tuleb välja, miks programmeerijad 16-süsteemi armastavad: üks number on täpselt neli bitti, nii et baidi saab pooleks lõigata ja mõlemad pooled on kohe näha.

Sellist arvudest koosnevat programmi nimetatakse **masinkoodiks**. See on ainus keel, millest protsessor aru saab. Programm, mis liidab 5 ja 3 ning näitab tulemust, on mälus neli baiti:

Aadress	Mälus	Tähendab
0	`1E`	`LOAD 14`: võta A-sse pesa 14 sisu
1	`2F`	`ADD 15`: liida pesa 15 sisu juurde
2	`50`	`OUT`: näita A-d
3	`00`	`HALT`: lõpp
14	`05`	arv 5 (andmed, mitte käsk)
15	`03`	arv 3

> **Pane tähele:** pesas 14 olev `05` ja pesas 2 olev `50` on mälu jaoks täpselt samasugused baidid. Käsuks muutub bait alles siis, kui PC sinna jõuab ja protsessor selle käsuna läbi loeb. Kui programm kogemata andmete peale jookseb, proovib protsessor neid käskudena täita. Tulemus on tavaliselt naljakas või katastroofiline.

Too, dekodeeri, täida

Protsessor teeb kogu oma elu ühte ja sama tsüklit. Igal ringil on kolm sammu:

- **Too** (ingl *fetch*): loe mälust bait aadressilt, kuhu PC näitab. Liiguta PC ühe võrra edasi;
- **Dekodeeri** (*decode*): vaata, mis käsk see on. Siin töötavad väravad, mis bitid lahti harutavad;
- **Täida** (*execute*): tee käsk ära. Liitmise teeb sama liitja, mille eelmisel nädalal ehitasime.

Vaatame ülemist programmi samm-sammult. Alguses on A = 0 ja PC = 0.

Takt	PC	Toodud	Mida tehti	A	Väljund
1	0	`1E`	LOAD 14: A-sse 5	5	
2	1	`2F`	ADD 15: 5 + 3	8	
3	2	`50`	OUT: näitab 8	8	8
4	3	`00`	HALT: peatub	8	8

Mängus näed sama tabelit elusalt: vajuta „Samm“ ja protsessor kirjutab iga kolme sammu kohta, mida ta tegi.

Hüpped: tsüklid ja otsused

Kui käsud tuleks alati järjest, jookseks iga programm ühe korra läbi ja lõpetaks. Päris programmid kordavad asju ja teevad otsuseid. Selleks on hüpped.

`JUMP x` paneb PC-sse x-i. Järgmine käsk ei tule enam järgmisest pesast, vaid sealt, kuhu hüppasime. Nii saab programm minna tagasi algusesse ja sama asja uuesti teha. See on **tsükkel**.

`JZ x` hüppab ainult siis, kui A on null. See on **otsus**: kui midagi kehtib, mine sinna, muidu jätka. Iga `if`, `for` ja `while`, mida kunagi programmeerides kirjutad, muutub protsessoris lõpuks selliseks hüppeks.

```
0  ADD 15     ; A = A + 1      (pesas 15 on 1)
1  OUT        ; näita A-d
2  JUMP 0     ; mine tagasi algusesse
```

See kolmerealine programm loeb 1, 2, 3, 4… ja ei lõpe kunagi. Mängus on see 5. ülesanne. 6. ülesandes paned ta kümnel peatuma, ja siis läheb vaja `JZ`-i.

Takt ja gigahertsid

Iga tsükli sammu juhib **kell**: elektriline signaal, mis lööb ühtlases tempos nagu metronoom. Iga löök on üks **takt**. Mängus näed taktiloendurit. Kui vajutad „Käivita“, teeb mänguprotsessor umbes kolm takti sekundis, et jõuaksid silmaga järgi.

Nüüd tead lõpuks, mida tähendas PCPartPickeris number 4,5 GHz. Giga on miljard, herts on kord sekundis. 4,5 GHz protsessoril lööb kell **4,5 miljardit** korda sekundis. Mängu 6. ülesanne võttis 70 takti. Sinu silme all kulus selleks umbes 20 sekundit, päris protsessoril kuluks 16 nanosekundit.

Päris protsessor on veel kiirem, kui gigahertsid ütlevad. Tal on mitu **tuuma** ja iga tuum on nagu eraldi protsessor, mis töötab oma programmiga. Iga tuum suudab ühe taktiga teha mitu käsku korraga.

Miks keegi masinkoodi ei kirjuta?

Proovi mängus korrutada ja saad aru. Kaksteist rida selleks, et arvutada 4 × 3! Esimesed programmeerijad kirjutasid tõesti arve, kuid kiiresti mõeldi välja, kuidas iseennast aidata. Sama programm kolmel tasemel:

Tase	Kuidas välja näeb	Kes sellest aru saab
Masinkood	`1E 2F 50 00`	protsessor
Assembler	`LOAD 14`, `ADD 15`, `OUT`, `HALT`	inimene, kellel on käsutabel käepärast. Programm nimega *assembler* teeb sõnadest arvud. Mängus teeb seda nupp „Lae mällu“
Python	`print(5 + 3)`	peaaegu igaüks. Python teeb sellest ise masinkoodi

Kui kirjutad Pythonis üheainsa rea, teeb arvuti sellest sadu või tuhandeid masinkoodi käske. Protsessor näeb ainult neid. Tema jaoks pole kunagi olnud Pythonit, JavaScripti ega Minecrafti. On ainult baidid mälus, PC ja tsükkel: too, dekodeeri, täida.

Sõnastik

Sõna	Tähendus
aadress	mälupesa number
register	protsessori enda pisike ja väga kiire mälu. Meie mängus on üks: A
PC	*program counter*: aadress, kust tuleb järgmine käsk
käsk	üks samm, mida protsessor oskab teha, näiteks LOAD või ADD
masinkood	programm arvudena, ainus keel, mida protsessor mõistab
assembler	masinkood sõnadena: LOAD 14 asemel 1E
hüpe	käsk, mis muudab PC-d. Sellest tulevad tsüklid ja otsused
takt	üks kella löök. Gigaherts = miljard takti sekundis

Pildid ja heli arvudena

10IT · Sissejuhatus infotehnoloogiasse · 8. tund · konspekt

Selle konspekti lugemiseks ei pea sa midagi ette teadma. Kõik sõnad selgitan ära siis, kui neid esimest korda vaja läheb, ja arvutused teeme samm-sammult läbi. Lahenda „Proovi ise“ kastid ja võrdle vastust konspekti lõpus olevatega. Mängi samal ajal veebilehel **Pildilaborit**.

Pärast seda tundi oskad:
• kirjutada mustvalge piksli rea arvudena ja RGB-värvi koodina;
• arvutada pakkimata pildi andmemahu;
• arvutada pakkimata heli andmemahu.

Need kolm ongi kõige tähtsamad. Video osa on lisateema.

Kuidas foto arvutisse mahub?

Teed telefoniga pildi koolisööklast. Sekundi pärast on see sõbra telefonis teises linnas. Kaablit mööda ei saa aga saata supitaldrikut. Isegi mitte värve.

Arvuti teab ainult arve. Nii et kuskil peab pilt arvudeks muutuma, ja sama juhtub laulu ning videoga. Just seda trikki me täna lahti võtamegi. Pärast seda oskad ise välja arvutada, kui palju ruumi üks pakkimata pilt võtab.

Neli sõna, mida läheb vaja

Kui oled neid varem kuulnud, on see kiire meeldetuletus. Kui mitte, siis siit saad kõik vajaliku.

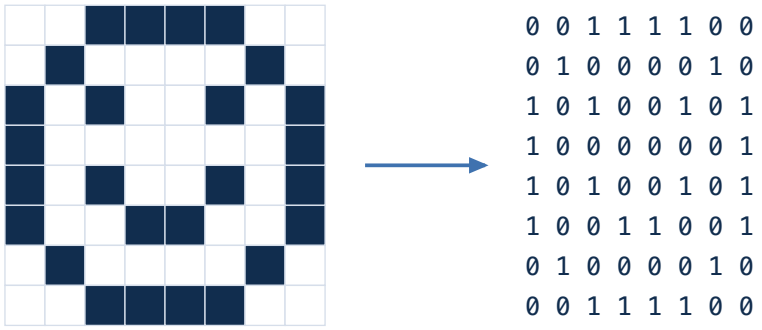
Sõna	Mis see on
bitt	Väikseim infokogus arvutis. Üks lüliti, mis on kas väljas (0) või sees (1).
bait	8 bitti koos. Ühe baidiga saab kirjutada arve 0 kuni 255, kokku 256 eri väärtust.
KB, MB, GB, TB	Nendes materjalides kasutame kümnendühikuid: 1 KB = 1000 baiti, 1 MB = 1 000 000 baiti, 1 GB = 1000 MB ja 1 TB = 1000 GB.
kuueteist-kümnendsüsteem	Lühidalt 16-süsteem. Arvude kirjutamise viis, kus numbreid on kuusteist: 0–9 ja siis A, B, C, D, E, F. A tähendab 10 ja F tähendab 15. Ühe baidi saab kirjutada kahe sellise märgiga.

Pilt on ruudustik

Suurenda telefonis mõnda pilti nii palju, kui saad. Mingil hetkel muutub see ruuduliseks, nagu matemaatikavihik, kus iga ruut on ühte värvi värvitud. Sellist ruutu nimetatakse **piksliks**. Sõna tuleb inglise keelest: *picture element*, pildi osake.

Rohkem pole seal midagi. Ükskõik milline foto on lihtsalt väga palju väikseid ühevärvilisi ruute.

Võtame kõige lihtsama juhtumi. Siin tähendab mustvalge pilt pilti, mille piksel on kas täiesti must või täiesti valge. Halltoone selles näites ei ole. Igal pikslil on seega kaks võimalust ja selleks piisab ühest bitist. Lepime kokku, et 1 on must ja 0 on valge. Siin on 8 pikslit lai ja 8 pikslit kõrge pilt:



Vasakul see, mida sina näed. Paremal see, mida arvuti tegelikult hoiab.

Vaata ülemist rida: `0 0 1 1 1 1 0 0`. Kaks valget, neli musta, kaks valget. Võrdle joonisega ja leia need ruudud üles. Kui saad sellest reast aru, oled pildifaili põhiidee juba kätte saanud.

Ja pane tähele üht veidrat asja: arvuti ei tea, et see on naerunägu. Tal on lihtsalt arvud. Pilt tekib alles siis, kui keegi need kokkuleppe järgi ruudustikuks joonistab.

Kui palju ruumi see pilt võtab? Arvutame:

```
1. samm: pikslite arv     8 × 8 = 64 pikslit
2. samm: bittide arv      64 pikslit × 1 bitt = 64 bitti
3. samm: baitide arv      64 : 8 = 8 baiti       (sest 1 bait = 8 bitti)
```

Proovi ise 1. Mustvalge pilt on 16 pikslit lai ja 8 pikslit kõrge. Mitu pikslit, mitu bitti ja mitu baiti see on? Ava siis Pildilaboris vaheleht **Bitipilt** ja klõpsa ruutudel: näed, kuidas read arvudeks muutuvad.

Kuidas rida 16-süsteemi kirjutada

Kaheksat nulli ja ühte on tülikas välja kirjutada. Seepärast kirjutavad programmeerijad baidi tavaliselt kahe 16-süsteemi märgina. See käib nii: jaga 8 bitti pooleks ja vaata kumbki pool allolevast tabelist järele.

Bitid	Märk	Bitid	Märk	Bitid	Märk	Bitid	Märk
0000	0	0100	4	1000	8	1100	C
0001	1	0101	5	1001	9	1101	D
0010	2	0110	6	1010	A	1110	E
0011	3	0111	7	1011	B	1111	F

Võtame naerunäo ülemise rea:

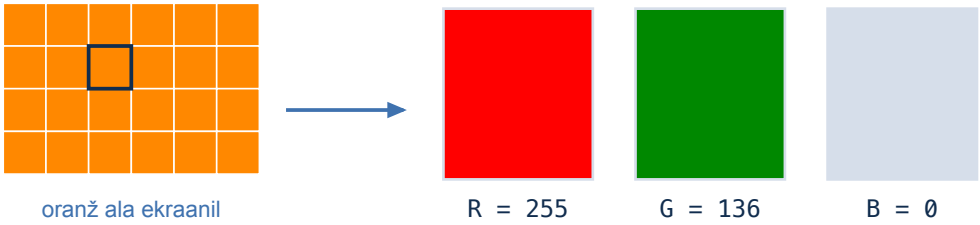
```
00111100          kirjutame kaheks pooleks
0011 | 1100       vaatame tabelist: 0011 on 3 ja 1100 on C
3C                vastus
```

Proovi ise 2. Kirjuta 16-süsteemis read `01100110` ja `11110000`.

Värvid: punane, roheline, sinine

Värvipildis ühest bitist ei piisa. Selles tunnis kasutame RGB-mudelit: RGB-värvipildis kirjeldame iga piksli värvi kolme arvuga, punase (R, ingl *red*), rohelise (G, *green*) ja sinise (B, *blue*) väärtusega.

Lihtsustatult võib tüüpilist ekraani ette kujutada nii, et iga piksel koosneb kolmest pisikesest lambist: punasest, rohelisest ja sinisest. Kaugelt silm neid enam eraldi ei erista ja näeb segu. Iga lambi ereduse jaoks on üks bait, seega arv 0-st (lamp ei põle) kuni 255-ni (põleb täie jõuga).



Üks oranž piksel: punane lamp täiega, roheline umbes poole peal, sinine ei põle.

Värv	R	G	B	16-süsteemis
must (kõik lambid kustus)	0	0	0	`#000000`
valge (kõik täiega)	255	255	255	`#FFFFFF`
punane	255	0	0	`#FF0000`
kollane (punane + roheline)	255	255	0	`#FFFF00`
oranž	255	136	0	`#FF8800`

Veebilehtede värvikood, näiteks `#FF8800`, ongi need samad kolm baiti 16-süsteemis: `FF` on punane, `88` roheline, `00` sinine. Kuidas tavalisest arvust selline kood saada? Jaga 16-ga. Täisosa annab esimese märgi ja jääk teise:

```
136 : 16 = 8, jääk 8     märgid on 8 ja 8    →  88
255 : 16 = 15, jääk 15   15 on F, seega      →  FF
 10 : 16 = 0, jääk 10    10 on A, seega      →  0A
```

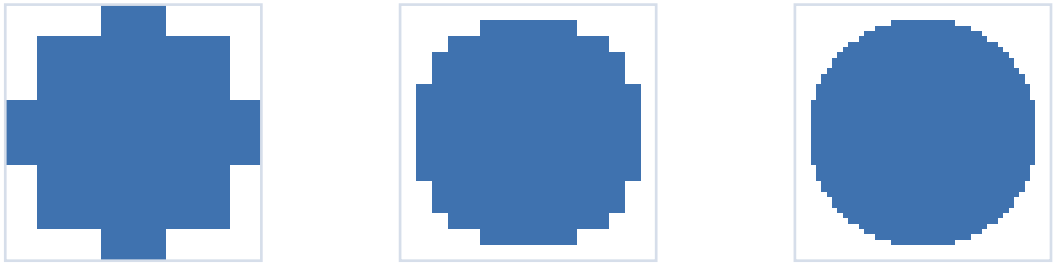
Vastupidi käib nii: korruta esimese märgi arvväärtus 16-ga ja liida teise märgi arvväärtus. `88` on 8 × 16 + 8 = 136.

Kui palju värve nii kokku saab? Punasel on 256 võimalust, rohelisel 256 ja sinisel 256, ja neid võib vabalt kombineerida: 256 × 256 × 256 = 16 777 216. Üle 16 miljoni värvi. Kahe naabervärvi vahet enamik inimesi ei näegi.

> **Proovi ise 3.** Kirjuta värvikoodina RGB (0, 0, 255) ja RGB (32, 0, 255). Proovi see järele vahelehel **Värv ja resolutsioon**, kus saad liugureid liigutada ja lampe näha.

Resolutsioon: mida rohkem ruute, seda teravam

Resolutsioon ehk pikslimõõtmed ütleb, mitu pikslit on pildil laiuses ja kõrguses. Täis-HD ekraanil on 1920 × 1080: 1920 pikslit laiuses ja 1080 kõrguses. Sama ring näeb eri resolutsioonis välja nii:



Sama ring. Mida rohkem piksleid, seda vähem on trepiastmeid näha.

Teravus maksab aga ruumi. Nendes ülesannetes kasutame pakkimata RGB-pilti, milles iga piksel võtab 3 baiti. Arvutame ainult pikslite andmemahtu; faili lisainfot ei arvesta.

Pakkimata pildiandmete maht = laius × kõrgus × baite ühes pikslis
RGB-pildil on pikslis 3 baiti. Baitidest megabaitide saamiseks jaga 1 000 000-ga.

Täis-HD pilt samm-sammult:

```
1. samm: pikslite arv    1920 × 1080 = 2 073 600 pikslit
2. samm: baitide arv     2 073 600 × 3 = 6 220 800 baiti
3. samm: megabaidid      6 220 800 : 1 000 000 ≈ 6,2 MB
```

Proovi ise 4. RGB-pilt on 1000 pikslit lai ja 500 kõrge. Kui suur on selle pakkimata andmemaht baitides ja megabaitides?

Nüüd mõistatus. Telefon teeb 12-megapikslise pildi. *Mega* tähendab miljonit: 12 000 000 × 3 baiti = 36 000 000 baiti ehk 36 MB. Nii palju võtaksid selle pakkimata RGB-andmed. Pakitud fotofail võib olla näiteks 3 MB. Kuhu ülejäänu kadus?

Pakkimine: kuidas 36 MB-st võib saada 3 MB

Oletame, et kirjutad poenimekirja. Sa ei kirjuta ju „õun, õun, õun, õun, õun“, vaid „5 õuna“. Info on sama, kirja läheb vähem. See ongi **pakkimine**: sama info kirjutatakse vähemate baitidega.

Pildil võib see välja näha nii: sinine taevas on tuhat täpselt sama värvi pikslit järjest. Tuhande piksli asemel saab kirjutada „1000 korda sinine“ ja pildi saab pärast täpselt samasugusena tagasi. Sellist pakkimist, kus midagi ära ei kao, nimetatakse **kadudeta** pakkimiseks. See näide näitab kadudeta pakkimise põhiideed. Ka PNG kasutab kadudeta pakkimist, kuid selle tegelik pakkimisviis on keerukam.

JPEG on julgem. Ta viskab päriselt infot minema, eelkõige pisikesi erinevusi, mida silm kergesti ei märka. Foto võib välja näha peaaegu samasugune, aga fail olla palju väiksem. See on **kadudega** pakkimine. Äravisatut enam tagasi ei saa.

JPEG-pildi kvaliteet võib halveneda, kui pilti muudetakse ja seejärel uuesti JPEG-vormingus salvestatakse. Faili muutmata kopeerimine või saatmine selle kvaliteeti ei vähenda.

Failivorming	Pakkimine	Sobib
BMP	tavaliselt ei paki	tänapäeval harva, fail on suur
PNG	kadudeta	ekraanipildid, logod, tekst pildil: seal, kus on teravad servad
JPEG	kadudega	fotod
WebP, AVIF	mõlemat moodi	veebilehed: uuemad vormingud, failid sageli väiksemad

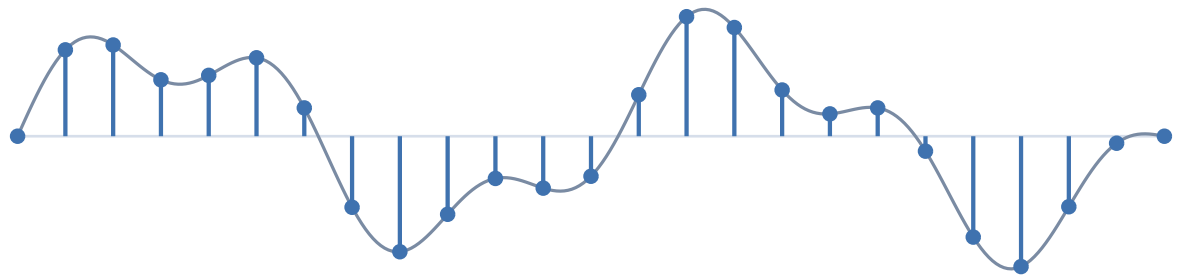
Järgmises tunnis teeme pakkimist töötoas ise, nii et praegu piisab sellest põhiideest.

Heli: õhu värisemine

Löö kitarrikeelt. Keel võngub ehk väriseb kiiresti edasi-tagasi, tõukab õhku ja see liikumine jõuab su kõrva sisse. Heli ongi õhu värisemine. Mikrofon teeb sama asja vastupidi: õhk liigutab tema sees pisikest plaati ja sellest tekib elektrivool, mis tõuseb ja langeb koos heliga.

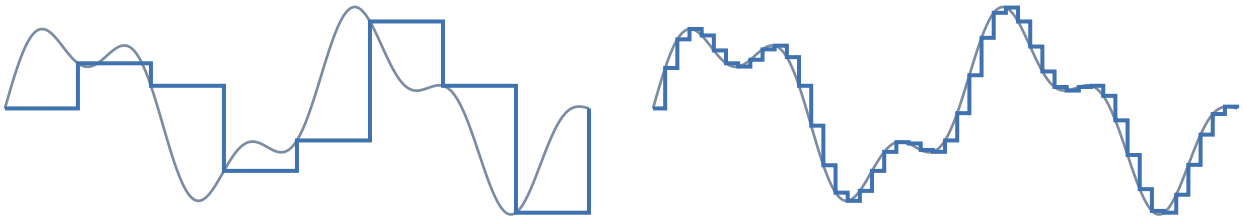
Kui selle tõusmise ja langemise joonistad, saad laine. Laine on sujuv, aga arvuti oskab hoida ainult üksikuid arve. Mida teha?

Mõtle ilmajaamale. Õhutemperatuur muutub kogu aeg, aga jaam kirjutab selle üles iga tunni tagant. Nende arvude järgi saab päeva graafiku uuesti joonistada. Arvuti teeb helilainega sama, ainult et mitte kord tunnis, vaid kümneid tuhandeid kordi sekundis. Seda mõõtmist nimetatakse **diskreetimiseks** (ingl *sampling*).



Hall joon on päris helilaine. Iga sinine täpp on üks mõõtmine: üks arv, mille arvuti salvestab.

Mõõtma peab piisavalt tihti, et helis olevad kõige kõrgemad sagedused kirja saada. Kui mõõdad liiga harva, kaovad kiired võnked ära ja heli muutub tuhmiks, nagu vana telefonikõne.



vähe mõõtmisi palju mõõtmisi

Hall on päris laine. Sinine astmeline joon aitab mõõtmistulemusi näha. Tegelik heli taastamine on keerukam.

Järgmises näites kasutame CD-heli parameetreid: 44 100 mõõtmist sekundis kanali kohta, 2 baiti mõõtmise kohta ja 2 kanalit. Igal arvul on oma põhjus:

Arv	Mida tähendab	Miks just nii
44 100	mõõtmist sekundis kanali kohta	Inimene kuuleb kuni umbes 20 000 võnget sekundis ja ühe võnke kirjeldamiseks on vaja vähemalt kaht mõõtmist.
2 baiti	ühe mõõtmise kohta	See on 16 bitti, millega saab eristada 65 536 erinevat signaalitaset.
2 kanalit	stereo	Eraldi vasaku ja parema kanali helisignaal. Üks kanal oleks mono.

Pakkimata heli andmemaht = mõõtmisi sekundis × baite mõõtmises × kanaleid × sekundeid

Üks minut CD-heli samm-sammult:

```
1. samm: üks sekund, üks kanal   44 100 × 2 = 88 200 baiti
2. samm: üks sekund, stereo      88 200 × 2 = 176 400 baiti
3. samm: üks minut               176 400 × 60 = 10 584 000 baiti ≈ 10,6 MB
```

Pakkimata kolmeminutiline laul võtaks seega üle 30 MB. MP3 ja teised sarnased vormingud kasutavad jälle kadudega pakkimist ja viskavad ära helid, mida kõrv kergesti ei kuule, näiteks vaikse heli kohe valju heli kõrval. Tulemus võib olla näiteks umbes 1 MB minutis.

> **Proovi ise 5.** Kui suur on 10 sekundit pakkimata CD-heli stereos? Vaata siis vahelehel **Heli**, mis lainega juhtub, kui mõõtmiste arvu liuguriga vähendad.

Lisateema. Video: palju pilte järjest

Video on nagu lehenurga-multikas: palju pilte, mida näidatakse nii kiiresti, et silm näeb liikumist. Ühte pilti videos nimetatakse **kaadriks**. Sageli on sekundis 30 kaadrit. Arvutame pakkimata täis-HD video pildiandmete mahu. Heli siin ei arvesta. Täpne baitide arv jääb arvutuse lõpuni alles ja ümardame ainult lõppvastuse:

```
1. samm: üks kaader (vaata arvutust eespool)   6 220 800 baiti
2. samm: üks sekund    6 220 800 × 30 = 186 624 000 baiti
3. samm: kaks tundi    2 tundi on 7200 sekundit
                       186 624 000 × 7200 = 1 343 692 800 000 baiti ≈ 1,3 TB
```

Pakitud kahetunnine film võib olla näiteks umbes 6 GB. Kuidas? Video pakkimine on väga kaval: kaks järjestikust kaadrit on sageli peaaegu samad, nii et salvestatakse enamasti ainult see, mis muutus. Kui uudistes liigub vaid diktori suu, siis on muutusi vähe.

> **Proovi ise 6.** Kui palju ruumi võtaksid 10 sekundi pakkimata täis-HD video pildiandmed?

Sõnastik

Sõna	Tähendus
piksel	pildi väikseim ruut, mis on ühte värvi
resolutsioon	pikslimõõtmed: mitu pikslit on pildil laiuses ja kõrguses, näiteks 1920 × 1080
RGB	värvimudel, mis kirjeldab värvi punase, rohelise ja sinise väärtusega
andmemaht	mitu baiti andmed võtavad
failivorming	kokkulepe, kuidas andmed failis on kirjas, näiteks PNG või JPEG
pakkimine	sama info kirjutamine vähemate baitidega
kadudeta	pakkimine, kus midagi ei lähe kaotsi (PNG, ZIP)
kadudega	pakkimine, mis viskab osa infot jäädavalt ära (JPEG, MP3)
diskreetimine	helilaine mõõtmine kindlate vahedega
mono, stereo	üks helikanal; eraldi vasak ja parem helikanal
kaader	üks pilt videost

„Proovi ise“ vastused

1. 16 × 8 = 128 pikslit. Igaüks on 1 bitt, seega 128 bitti. 128 : 8 = 16 baiti.
2. `0110` on 6 ja `0110` on 6, seega `66`. `1111` on F ja `0000` on 0, seega `F0`.
3. `#0000FF` ja `#2000FF`. 32 : 16 = 2, jääk 0, seega on punane `20`.
4. 1000 × 500 = 500 000 pikslit. 500 000 × 3 = 1 500 000 baiti ehk 1,5 MB.
5. Üks sekund stereot on 176 400 baiti. 176 400 × 10 = 1 764 000 baiti ≈ 1,8 MB.
6. 186 624 000 × 10 = 1 866 240 000 baiti ≈ 1,9 GB.

Pakkimine

10IT · Sissejuhatus infotehnoloogiasse · 9. tund · konspekt

See konspekt selgitab kõik algusest peale ja seda saab lugeda ka tunnist puududes. Lahenda „Proovi ise“ kastid, vastused on lõpus. Veebilehel on **Pakkimislabor**, kus saad kõike päris failidega järele proovida. Konspekti lõpus on kirjas, mida tunnis samm-sammult teeme.

Pärast seda tundi oskad:
• pakkida ja lahti pakkida RLE-ga;
• arvutada, mitu korda andmemaht väheneb ja mitu protsenti säästsid;
• selgitada, millal sobib kadudeta ja millal kadudega pakkimine.

Poenimekiri

Sul on vaja poest 12 õuna tuua. Sa ei kirjuta paberile kaksteist korda „õun“, vaid „12 õuna“. Selle pisikese trikiga oledki avastanud pakkimise.

Pakkimine tähendab, et sama info kirjutatakse vähemate baitidega. Üks bait koosneb 8 bitist. Märgi salvestamiseks vajalik baitide arv sõltub kodeeringust: näiteks UTF-8 kodeeringus võtab a ühe baidi ja õ kaks baiti. 1 MB on 1 000 000 baiti.

Eelmises tunnis arvutasime, et 12-megapikslise pildi pakkimata RGB-andmed võtaksid 36 MB, aga pakitud fotofail võib olla näiteks 3 MB. Ilma pakkimiseta mahuks telefoni palju vähem pilte ja video vaatamine netist oleks palju raskem. Täna saad teada, kuidas see käib, ja proovid järele.

Pakkimisviis nr 1: loe järjest samad kokku

Mõtle pildile, kus iga piksel (pildi väikseim ruut) on kas valge või must. Eelmises tunnis piisas kahevärvilise pildi ühe piksli jaoks ühest bitist. Siin kasutame RLE õppimiseks lihtsustatud esitust: iga piksel on kirjas ühebaidise märgina V (valge) või M (must). Üks rida kümnest pikslist:

```
V V V V V M M M V V        10 pikslit = 10 baiti
```

Selle asemel, et iga piksel eraldi üles kirjutada, loeme kokku, mitu ühesugust on järjest: viis valget, kolm musta, kaks valget. Kirjutame `5V 3M 2V`. Meie mudelis salvestatakse korduste arv ühe baidina ja värvitähis teise baidina. Paare on kolm, nii et kokku 6 baiti. Kümne asemel! Ka paar `12V` võtab kaks baiti. Näidetes olevad tühikud on ainult lugemise hõlbustamiseks.



Sama rida: pakkimata 10 baiti, pakituna 6 baiti.

Seda viisi nimetatakse **RLE**-ks (ingl *run-length encoding*). Eesti keeles võiks öelda „järjendite pakkimine“, kus järjend on rida ühesuguseid asju.

Lahtipakkimine on sama lihtne: kirjuta iga paari järgi pikslid uuesti välja. `2M 3V` tähendab kaks musta ja kolm valget, ehk `MMVVV`. Tulemus on täpselt sama, mis enne pakkimist, midagi ei kadunud. See on **kadudeta** pakkimine.

Proovi ise 1. Paki rida `MMMMVVVVVV`. Mitu baiti see võtab? Paki lahti `2V 3M 1V`. Harjuta Pakkimislabori vahelehel **RLE**, seal kontrollib arvuti vastuse kohe ära.

Kui hästi see töötas?

Pakkimise tulemust mõõdetakse kahel viisil. Mõlemad arvutused põhinevad jagamisel.

```
Mitu korda väiksem?       enne : pärast
                          10 baiti : 5 baiti = 2 korda

Mitu protsenti säästsid?  (enne − pärast) : enne × 100
                          (10 − 5) : 10 × 100 = 50 %
```

Kui 36 MB-st saab 3 MB: 36 MB : 3 MB = 12 korda väiksem. Säästetud on (36 − 3) : 36 × 100 ≈ 92 %. Kui võrdled eri pikkusega faile, on protsent parem mõõt kui säästetud baitide arv.

Proovi ise 2. Fail oli 5 MB, pärast pakkimist 1 MB. Mitu korda väiksem ja mitu protsenti säästetud?

Aga mis siis, kui järjest ühesuguseid pole?

Proovi nüüd malelauda, kus värvid vahelduvad iga ruudu järel:



Kaheksa pikslit, kaheksa paari. Pakitud kuju on suurem kui algne!

Kaheksast baidist sai kuusteist. Andmemaht *kasvas* kaks korda. See pole programmiviga. RLE elab kordustest ja kui korduseid pole, pole ka millestki kokku hoida.

See on tähtis mõte, mis tuleb täna veel mitu korda tagasi: **ükski kadudeta pakkija ei suuda iga faili väiksemaks teha.**

Pakkimisviis nr 2: sõnastik

RLE aitab ainult siis, kui samad asjad on kohe üksteise kõrval. Tekstis on sageli teisiti. Vaata seda lauset. Selles näites loeme märke, sealhulgas tühikuid ja kirjavahemärke. See ei ole faili täpne baitide arv.

```
kass istub, kass sööb, kass magab          33 märki
```

Järjest ühesuguseid tähti siin pole, aga sõna „kass“ kordub. Teeme väikese sõnastiku: 1 tähendab sõna „kass“.

```
sõnastik: 1=kass                            6 märki
tekst:    1 istub, 1 sööb, 1 magab         24 märki
kokku                                      30 märki
```

Pane tähele, et sõnastik tuleb ka kaasa pakkida, muidu ei saa keegi teksti lahti pakkida. Seepärast võitsime siin ainult 3 märki. Pikas tekstis, kus sama sõna on sada korda, on võit palju suurem. Mõtle laulule, kus refrään kordub: piisab, kui refräänist on üks koopia ja mujale kirjutad „nüüd tuleb refrään“.

Sarnast ideed kasutavad levinud kadudeta vormingud gzip ja ZIP. Natuke kavalamalt küll: eraldi sõnastiku asemel võib seal olla kirjas „mine 16 märki tagasi ja kopeeri sealt 4 märki“. Idee on sama: ära kirjuta kaks korda seda, mis on juba olemas. Programmis või tekstis ei tohi ükski täht kaotsi minna, seega sobib siia just kadudeta pakkimine.

Proovi ise 3. Paki sõnastikuga lause `tere tere tere Mari`. Mitu märki oli enne ja mitu pärast, kui ka sõnastik kokku lugeda?

Miks juhuslikku asja on raske pakkida?

Kujuta ette, et pead sõbrale telefonis ette lugema tähed `xqf7kP2z`. Lühemalt seda öelda ei saa, pead kõik ükshaaval ära ütlema. Mustrit pole, nii et pole ka midagi lühendada.

Sarnane lugu on juba pakitud failiga. Juba pakitud failis on tavaliselt vähem lihtsalt ärakasutatavaid kordusi. Seepärast ei tee ZIP JPEG-pilti tavaliselt palju väiksemaks ja ZIP-i pakkimine teise ZIP-i sisse enamasti ei aita.

Pakitud andmetele lisandub ka vormingu lisainfo. Seetõttu võib väga lühike tekst pakkimisel hoopis suureneda. Seda näed vahelehel **gzip-labor**. Selles laboris pakib brauser teksti gzip-vormingus ja näitab baitide arvu enne ja pärast. ZIP on teine levinud pakkimisvorming.

Pakkimisviis nr 3: viska ära, mida silm kergesti ei näe

Foto on keerulisem. Fotol võib olla palju väikeseid värvierinevusi, mistõttu lihtne RLE ei pruugi hästi töötada. JPEG lähenebki teisest otsast: kas inimesel on vaja iga väikest erinevust?

Kujuta ette, et kirjeldad telefonis sõbrale pilti taevast. Sa ei hakka ütlema iga ruudu täpset värvi, vaid ütled: „see nurk on helesinine“. Sõber saab enam-vähem sama pildi. JPEG jagab pildi väikesteks ruutudeks (8 × 8 pikslit) ja lihtsustab igaühes väikeseid erinevusi.



Lihtsustatud näide väikeste heleduseerinevuste eemaldamisest; see ei näita JPEG-i täpset tööalgoritmi.

See on **kadudega** pakkimine. Infot läheb päriselt kaotsi ja seda ei saa kuidagi tagasi. Salvestamisel saad valida kvaliteedi: kõrgem kvaliteet tähendab vähem ära visatud infot ja tavaliselt suuremat faili.

Siit tuleb ka see, miks tekst JPEG-is võib halvasti välja näha. Teravate mustade tähtede ümber võib tekkida plekiline aura. Seepärast on ekraanipildi jaoks PNG sageli parem.

JPEG-pildi kvaliteet võib halveneda, kui pilti muudetakse ja seejärel uuesti JPEG-vormingus salvestatakse. Faili muutmata kopeerimine või saatmine selle kvaliteeti ei vähenda. Vaata seda vahelehel **JPEG-labor** nupuga „Salvesta uuesti“, mis lõikab pildilt iga kord serva ära ja salvestab uuesti.

Mida millal kasutada?

Asi	Vali	Miks
foto	JPEG	Väikesed kaod ei paista tavaliselt välja.
ekraanipilt, logo, joonis	PNG	Palju ühesugust värvi pikslit, teravad servad ei tohi plekiliseks minna.
tekst, kood, programmid	ZIP, gzip	Ükski märk ei tohi kaduda.
muusika	MP3, AAC	Kadudega: väike kadu jääb tavaliselt kõrvale märkamatuks.
juba pakitud fail	ära paki	Sellest ei tule tavaliselt palju kasu.

Mida tunnis teeme

1. **RLE käsitsi.** Ava Pakkimislaboris vaheleht RLE ja tee ülesanded läbi. Täida töölehe 1. ja 2. osa.
2. **Päris gzip.** Vahelehel gzip-labor on kolm nuppu: korduv tekst, tavaline tekst ja juhuslikud märgid. Vajuta kõiki, kirjuta tulemused töölehe 3. ossa ja kleebi kasti ka oma tekst.
3. **Päris JPEG.** Vahelehel JPEG-labor vali foto moodi pilt, seejärel logo moodi pilt ja liiguta kvaliteeti. Kirjuta suurused töölehe 4. ossa. Soovi korral proovi oma pildiga, see ei lahku su arvutist.
4. **Selgita.** Täida töölehe viimane osa, tee kodus lõpuni, mis jäi pooleli, ja saada tööleht ära. Ülesanne 5c on lisaülesanne.

Sõnastik

Sõna	Tähendus
andmemaht	mitu baiti andmed võtavad
pakkimine	sama info kirjutamine vähemate baitidega
kadudeta	lahti pakkides saad täpselt sama faili tagasi (PNG, ZIP, gzip)
kadudega	osa infot visatakse jäädavalt ära (JPEG, MP3)
RLE	pakkimine, mis loeb kokku, mitu ühesugust asja on järjest: 5V 3M
sõnastik	nimekiri korduvatest tükkidest, millele saab tekstis lühidalt viidata
ZIP, gzip	levinud kadudeta pakkimisvormingud
kvaliteet	JPEG-i seadistus: mida kõrgem, seda vähem infot visatakse ära

„Proovi ise“ vastused

1. `4M 6V`, kaks paari ehk 4 baiti. Lahti pakituna `VVMMMV`.
2. 5 : 1 = 5 korda väiksem. (5 − 1) : 5 × 100 = 80 % säästetud.
3. Enne 19 märki. Sõnastik `1=tere` on 6 märki ja tekst `1 1 1 Mari` on 10 märki, kokku 16.